

A Self-Applicable Normaliser for a Small Dependent Theory Language

A C++23-oriented formal core

15 August 2026

Abstract

This note defines a small dependently typed theory language and a self-applicable normaliser for it. The construction uses de Bruijn syntax, a predicative universe hierarchy, bidirectional type checking, normalization by evaluation (NbE), and staged quotation. We state the central metatheoretic properties for the core: substitution, type preservation of reduction, normalization of well-typed terms, and soundness of the normaliser. Eight uses are described, including proof certificates, dependent data, staged metaprogramming, and normalization of reflected syntax. The final sections give a C++23 implementation sketch, a typed staging interface, additional lemmas, complexity considerations, and a testing strategy.

Contents

1	Scope and design constraints	2
2	The core language	2
2.1	Syntax	2
2.2	Universes and typing rules	2
3	Substitution and reduction	2
4	Normalization by evaluation	3
4.1	Semantic values	3
4.2	Reflection and reification	3
5	Staged self-application	3
6	Eight uses	4
6.1	Four simple uses	4
6.2	Four complex uses	4
7	Correspondence with the C++23 core	4
8	Limitations and extensions	4
9	Implementation sketch	4
10	Bidirectional checking	5
11	Corrected staging interface	5
12	Additional metatheoretic lemmas	5

13 Normalization algorithm and complexity	5
14 Testing strategy	6
15 Implemented C++23 prototype	6
15.1 Verification results	6
16 Conclusion	7

1 Scope and design constraints

The phrase “self-applicable” is potentially misleading. An untyped evaluator can be applied to itself, but an unrestricted term of type $\text{Type} : \text{Type}$ makes the usual dependent theory inconsistent. We therefore use two restrictions:

1. universes are predicative: $\text{Type}_i : \text{Type}_{i+1}$;
2. self-application is staged: source syntax is data, and only an explicitly unquoted code value is executed.

Thus the normaliser can process representations of normalisers without introducing an unrestricted self-typing paradox.

2 The core language

2.1 Syntax

Terms are represented with de Bruijn indices. The newest variable is index 0:

$$t ::= x_i \mid \text{Type}_i \mid (x : A) \rightarrow B \mid \lambda : A. t \mid t u \mid \text{quote}(()t) \mid \text{unquote}(()t).$$

The dependent function type may mention x in B . Ordinary functions are the special case in which B does not mention x . Contexts are lists $\Gamma = A_0, A_1, \dots$ with A_0 the type of x_0 .

2.2 Universes and typing rules

The universe rules are

$$\frac{\Gamma \vdash A : \text{Type}_i \quad \Gamma, A \vdash B : \text{Type}_j}{\Gamma \vdash (x : A) \rightarrow B : \text{Type}_{\max(i,j)}} \quad \frac{}{\Gamma \vdash \text{Type}_i : \text{Type}_{i+1}}.$$

The term rules include

$$\frac{\Gamma \vdash A : \text{Type}_i \quad \Gamma, A \vdash t : B}{\Gamma \vdash \lambda : A. t : (x : A) \rightarrow B} \quad \frac{\Gamma \vdash f : (x : A) \rightarrow B \quad \Gamma \vdash u : A}{\Gamma \vdash f u : B[u/x]}.$$

The beta rule is $(\lambda : A. t)u \rightarrow_\beta t[u/x]$. Eta-long normal forms are used by NbE.

3 Substitution and reduction

Let $\uparrow_d^k t$ add d to every index at least k ; under a binder the cutoff becomes $k + 1$.

Lemma 3.1 (Shifting composition). *For compatible cutoffs, $\uparrow_a^k (\uparrow_b^k t) = \uparrow_{a+b}^k t$.*

Proof. By structural induction on t . The variable case is arithmetic on whether the index is below the cutoff. Binder cases use the induction hypothesis at cutoff $k + 1$. $\square$

Lemma 3.2 (Substitution under binders). *If $\Gamma, A, \Delta \vdash t : B$ and $\Gamma \vdash u : A$, then capture-avoiding substitution $t[u/x]$ is well formed in $\Gamma, \Delta[u/x]$.*

Proof. Induct on the typing derivation. In binder cases, shift u before descending. Applications follow from the induction hypotheses and substitution for codomains. $\square$

Theorem 3.3 (Type preservation). *If $\Gamma \vdash t : A$ and $t \rightarrow_\beta t'$, then $\Gamma \vdash t' : A$.*

Proof. The only primitive reduction is beta. Inversion gives a typed abstraction and argument; the substitution lemma gives the type of the contracted body. Congruence preserves typing in surrounding terms. $\square$

4 Normalization by evaluation

4.1 Semantic values

For each context and type, semantic values are neutrals, closures, sorts, and syntax values:

$$\text{Val}_\Gamma(A) ::= \text{Neutral}(n) \mid \text{Closure}(\rho, t) \mid \text{Sort}(i) \mid \text{Syntax}(t).$$

Evaluation is written $\text{eval}_\rho(t)$. Variables read from environments; lambdas form closures; applications use `apply`; quotation produces syntax without evaluating its body; unquotation executes only a syntax value.

4.2 Reflection and reification

Reflection embeds a neutral into a value. Reification $\text{quote}_A(v)$ produces an eta-long normal form. At function types, a closure is applied to a fresh neutral and the result is reified. The normaliser is

$$\text{norm}(t) = \text{quote}_A(\text{eval}_\emptyset(t)).$$

Theorem 4.1 (Normalization). *If $\Gamma \vdash t : A$, then $\text{norm}(t)$ is defined and is an eta-long beta-normal form of type A .*

Proof. By induction on typing, using the logical relation indexed by types. The predicative hierarchy and absence of general recursion ensure that evaluation of well-typed terms is strongly normalizing. $\square$

Theorem 4.2 (Soundness of normalization). *If $\Gamma \vdash t : A$, then $\Gamma \vdash \text{norm}(t) : A$ and $t \equiv \text{norm}(t)$.*

Proof. Reification maps related values to well-typed normal forms. Reflection and reification are inverse up to definitional equality; at function types the proof applies both sides to a fresh variable. $\square$

5 Staged self-application

Let N be the implementation of `norm`. Its code is a syntax value $N = \text{quote}(()N)$. Self-application is $\text{selfNorm}(c) = \text{unquote}(()N) c$. The safety point is that N is data and is not executed merely because it appears in a term.

Proposition 5.1 (Staged self-correctness). *For every closed well-typed t , unquoting the quoted normaliser and applying it to t produces the same normal form as the host-level normaliser.*

Proof. Quotation preserves the abstract syntax of N , unquotation restores it, and the soundness theorem identifies the result with $\text{norm}(t)$. $\square$

6 Eight uses

6.1 Four simple uses

Beta reduction. $I = \lambda : A. \lambda x : A. x$ and $t = I a$ normalize to a .

Dependent result types. For a family $\text{Vec}(A, n)$, a head function has type $(n : \text{Nat}) \rightarrow (A : \text{Type}_0) \rightarrow \text{Vec}(A, n + 1) \rightarrow A$. The index rules out empty vectors.

Proof-term simplification. $(\lambda p : P. p)q$ normalizes to q without changing its type.

Staged code templates. $\text{unquote}(()\text{quote}(()\lambda : A. x))$ restores the original lambda after an explicit stage transition.

6.2 Four complex uses

Certified compiler passes. With an indexed syntax family $\text{Expr}(\tau)$, a pass $C : (\tau : \text{Ty}) \rightarrow \text{Expr}(\tau) \rightarrow \text{Expr}(\text{lower}(\tau))$ can carry a preservation proof. Normalization canonicalizes generated certificates.

Dependent protocol state machines. If $\text{Conn}(s)$ is a connection in state s , then $\text{send} : \text{Conn}(\text{Ready}) \rightarrow M \rightarrow \text{Conn}(\text{Waiting})$ prevents ill-ordered operations before normalization.

Reflected decision procedures. A checker $\text{check} : \text{Code}(P) \rightarrow \text{Decidable}(P)$ may normalize reflected syntax before inspecting it, provided recursive inspection is structurally bounded.

Typed metaprograms. A transformation $T : (A : \text{Type}_i) \rightarrow \text{Code}(A) \rightarrow \text{Code}(A)$ may quote, transform, and unquote its own code while preserving the indexed type.

7 Correspondence with the C++23 core

Mathematics	C++23
t	<code>Term</code>
x_i	<code>Term::var(i)</code>
Type_i	<code>Term::sort(i)</code>
Pi and lambda	<code>Term::pi</code> , <code>Term::lam</code>
substitution	<code>substitute</code>
type inference	<code>infer</code>
semantic value	<code>Value</code>
evaluation	<code>eval</code>
reification	<code>quoteValue</code>
normalization	<code>normalise</code>
staging	<code>quote</code> , <code>unquote</code>

8 Limitations and extensions

Adding unrestricted general recursion destroys total normalization. Inductive types require positivity and sound eliminators. Equality reflection may make conversion undecidable. A safe extension path is: natural numbers with structural recursion; identity types; positive inductive families; and a total meta-language for tactics and compilation.

9 Implementation sketch

The implementation uses immutable syntax trees and shared ownership:

```

using Index = std::uint32_t;
using Level = std::uint32_t;
struct Term;
using TermPtr = std::shared_ptr<const Term>;
struct Var { Index index; };
struct Sort { Level level; };
struct Pi { TermPtr domain, codomain; };
struct Lam { TermPtr domain, body; };
struct App { TermPtr function, argument; };
struct Quote { TermPtr body; };
struct Unquote { TermPtr code; };
struct Term {
    using Node = std::variant<Var,Sort,Pi,Lam,App,Quote,Unquote>;
    Node node;
    template<class T> explicit Term(T x) : node(std::move(x)) {}
};

```

Semantic values must distinguish closures, neutrals, sorts, and syntax values. A closure stores the environment in which its body was created. Application evaluates a closure by extending its environment, while application of a neutral creates a neutral application.

10 Bidirectional checking

The checker is divided into synthesis $\text{infer}(\Gamma, t)$ and checking $\text{check}(\Gamma, t, A)$. A lambda checks against an expected Pi type; variables, sorts, and applications synthesize. Conversion is implemented by evaluating and reifying both candidate types, then structurally comparing their eta-long normal forms.

11 Corrected staging interface

A typed staging interface is given by

$$\text{Code} : (A : \text{Type}_i) \rightarrow \text{Type}_i, \quad \text{quote}(:)A \rightarrow \text{Code}(A), \quad \text{unquote}(:)\text{Code}(A) \rightarrow A.$$

Unquotation is therefore type preserving. The normaliser may have type $(A : \text{Type}_i) \rightarrow \text{Code}(A) \rightarrow \text{Code}(A)$, while self-application remains an explicit stage transition rather than an inhabitant of an unrestricted self-function type.

12 Additional metatheoretic lemmas

Lemma 12.1 (Weakening). *If $\Gamma \vdash t : A$, then $\Gamma, B \vdash_0^0 t :_0^0 A$ for every well-formed B .*

Lemma 12.2 (Evaluation respects substitution). *For compatible environments, $\text{eval}_\rho(t[u/x]) = \text{eval}_{\rho, \text{eval}_\rho(u)}(t)$.*

Lemma 12.3 (Quotation preserves typing). *If $\Gamma \vdash t : A$, then $\Gamma \vdash \text{quote}((t)) : \text{Code}(A)$.*

Theorem 12.4 (Stage safety). *If $\Gamma \vdash c : \text{Code}(A)$, then $\Gamma \vdash \text{unquote}((c)) : A$.*

13 Normalization algorithm and complexity

For open terms, the empty environment is replaced by an environment of reflected fresh neutrals. The pipeline is: infer the type; reflect the context; evaluate; reify. Eta-long output makes definitional equality reducible to structural equality of normal forms. Strong normalization does not imply small output: dependent reification can duplicate neutral computations. Implementations

should therefore expose step, depth, and output-node limits and distinguish resource exhaustion from type errors.

14 Testing strategy

Tests should cover shifting and substitution, closure application, neutral applications, malformed applications, universe mismatches, invalid unquotation, and the invariant

$$\text{infer}(\Gamma, t) = A \implies \text{infer}(\Gamma, \text{normalise}(\Gamma, t)) = A.$$

Normalization should be idempotent on accepted terms: $\text{normalise}(\Gamma, n) = n$ whenever n is already eta-long and beta-normal.

15 Implemented C++23 prototype

The accompanying implementation, `normaliser.cpp`, follows the core directly. Terms are immutable shared trees whose node type is a C++23 variant containing variables, universes, explicit code types, Pi types, lambdas, applications, quotation, and unquotation. Semantic evaluation uses environments, closures, neutral values, and syntax values. Dependent codomains are instantiated with de Bruijn substitution; ordinary computation is performed by the NbE evaluator.

The central executable interface is:

```
T nbe_normalise(const Ctx& context, const T& term) {
    auto type = infer(context, term);
    return reify(eval_nbe(reflect(context), term), type);
}

T self_apply(const Ctx& context, const T& term) {
    auto quoted = quoted_normaliser();
    return unquote_code(quoted, context, term);
}
```

The quotation boundary is represented by typed `Code(A)` terms. The checker enforces

$$\text{quote}(()t) : \text{Code}(\text{type}(t)), \quad \text{unquote}(()c) : A \quad \text{when } c : \text{Code}(A).$$

The host-side `quoted_normaliser` is deliberately explicit: the normaliser is inert while quoted and is invoked only by `unquote_code`. This models the staged self-application boundary without adding an unrestricted `Type : Type` universe.

The implementation also checks predicative universe formation. In particular,

$$\text{infer}(\text{Type}_i) = \text{Type}_{i+1}$$

and the type of $(x : A) \rightarrow B$ is formed at the maximum universe level of A and B .

15.1 Verification results

The executable was compiled with:

```
g++ -std=c++23 -Wall -Wextra -pedantic -O2 normaliser.cpp -o normaliser
./normaliser
```

The test program verifies beta normalization, staged self-application, eta-long reification of both neutral functions and closures, universe successor and Pi-level formation, rejection of an ill-typed application, typed quotation, idempotence of normalization, and the de Bruijn substitution invariant. The observed output includes:

```
normal form: x0
staged self-application: x0
eta-long neutral: (lam (x1 x0))
eta-long closure: (lam (lam x0))
universe and rejection tests: PASS
typed quotation boundary: PASS
substitution invariant: PASS
```

The prototype remains intentionally small. It does not provide inductive types, user-defined recursion, equality reflection, or a separately verified compiler from the mathematical calculus to C++. Those are extensions beyond the strongly normalizing kernel implemented here.

16 Conclusion

The construction gives a precise meaning to a self-applicable normaliser: the normaliser is represented as typed code, and self-application is an explicit stage transition. NbE supplies efficient normalization, while the universe hierarchy and quotation boundary prevent unrestricted self-reference paradoxes. The result is small enough to implement directly in pure C++23, yet expressive enough to serve as a kernel for typed metaprogramming and proof-producing computation.